

k-Means as an ML Tool

2026-04-17

Introduction

k -means is usually taught as a stand-alone unsupervised clustering method, but in ML pipelines it is also a versatile feature-engineering primitive. Pre-clustering observations and using cluster membership (or distance-to-centroid) as new features can substantially improve downstream supervised models, especially when the underlying signal is non-linear and a clustering captures local structure that linear models miss. k -means is also the workhorse for vector quantisation, image segmentation, and customer-segmentation tasks.

Prerequisites

A working understanding of Euclidean distances, centroids, the within-cluster sum-of-squares objective, and the bias-variance trade-off in feature engineering.

Theory

k -means minimises the within-cluster sum of squares (WCSS):

$$\min_C \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2.$$

Lloyd's algorithm alternates assignment (each point to its nearest centroid) and update (each centroid to the mean of its assigned points) until convergence. The algorithm is non-convex; multiple random starts are needed to find a good local optimum. The number of clusters K is a hyperparameter chosen by the elbow, silhouette, or gap statistic methods.

For feature engineering, two derived quantities from a k -means fit are useful: - **Cluster membership** (one-hot or label): a categorical feature indicating which cluster each observation belongs to. - **Distance to each centroid**: K continuous features that often carry more predictive information than the cluster label alone.

Assumptions

Clusters are roughly spherical and equal-sized (the implicit Voronoi-cell geometry); features are scaled to comparable units; the chosen K matches a meaningful structure in the data.

R Implementation

```
set.seed(2026)
n <- 300
df <- data.frame(x1 = rnorm(n), x2 = rnorm(n), x3 = rnorm(n))

# Scale for kmeans
df_s <- scale(df)

# Fit with K=4 and use cluster membership as a feature
```

```

km <- kmeans(df_s, centers = 4, nstart = 20)
df$cluster <- factor(km$cluster)

# Distance to each centroid also becomes a feature
dist_to_centroids <- as.matrix(dist(rbind(df_s, km$centers)))
dist_features <- dist_to_centroids[1:n, (n + 1):(n + 4)]
colnames(dist_features) <- paste0("dist_c", 1:4)

head(cbind(df, dist_features))
km$tot.withinss

```

Output & Results

The augmented data frame contains cluster membership plus four distance-to-centroid features. The model's WCSS quantifies clustering tightness; lower values indicate more compact clusters.

Interpretation

A reporting sentence: “*k*-means with $K = 4$ on the standardised features reduced WCSS from 900 (one cluster, equivalent to total variance) to 620 (four clusters); using cluster membership and distance-to-centroid as additional features in the downstream XGBoost classifier improved cross-validated RMSE by 3 %.” Always state K , the scaling, and the number of random starts.

Practical Tips

- Scale features before *k*-means; unscaled features with different units dominate the Euclidean distance and bias the partition.
- Use `nstart = 20` or higher to escape local minima; *k*-means is non-convex and a single start often produces sub-optimal solutions.
- *k*-means-style feature engineering works best when clusters actually exist in the data; on truly random data it adds noise features and can hurt downstream performance.
- Distance-to-centroid features are often more informative than the raw cluster label because they capture the continuous geometry that the discrete label discards.
- For high-dimensional data, reduce dimensionality with PCA before *k*-means; Euclidean distances concentrate and clusters lose meaning in high dimensions.
- For categorical or mixed data, *k*-means does not apply directly; use *k*-modes or PAM with Gower distance instead.

R Packages Used

Base R `kmeans()` for the canonical implementation; `factoextra::fviz_cluster()` for ggplot-style cluster visualisation; `klaR::kmodes()` for categorical alternatives; `recipes::step_kmeans_features()` (via `embed`) for tidymodels-flavoured pipeline integration; `cluster::clara()` for *k*-medoids alternatives that scale to large datasets.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren't.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis